

UNIVERSIDAD DE COSTA RICA
SEDE PACÍFICO

ALGORITMOS Y ESTRUCTURAD DE DATOS

INFORMÁTICA
EMPRESARIAL

MATERIAL DIDÁCTICO



```
public class Nodo <T>{  
    private T dato;  
    private Nodo<T> siguiente;  
  
    public Nodo (T dato){  
        siguiente=null;  
        this.dato=dato;  
    }  
} // fin de la clase
```

ESTE TEXTO ESTÁ DIRIGIDO A QUIENES SE ESFUERZAN
POR APRENDER DÍA A DÍA, SIN IMPORTAR LAS
CONDICIONES ADVERSAS QUE IMPEREN

Tema 3

Tipos de datos abstractos

Karol Sánchez y Marcos Venegas

Conceptos básicos de los TDA

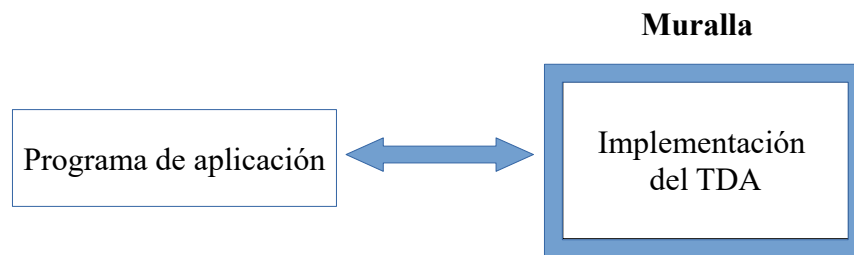
La abstracción de datos se puede definir como la técnica que nos permite crear nuevos tipos de datos que sean los más recomendados al programa que se está desarrollando, permitiendo que se reduzcan las líneas de código y que a la vez sea más flexible.

Cuando se crea un programa que dependa de la implementación definida para sus datos, tiene el problema que, si después se desea realizar una modificación a esa implementación, se van a generar problemas y para corregirlos serían muchos los cambios que se deban realizar. Este problema se puede eliminar utilizando tipos de datos abstractos en la implementación del programa.

Un tipo de dato abstracto, conocido como TDA, es un tipo de dato que lo crea el programador con todos los métodos necesarios según el problema que se quiera resolver. “Un TDA se compone de estructuras de datos y los procedimientos o funciones que manipulan esas estructuras de datos.” (Joyanes Aguilar, 2008)

Un TDA se puede definir como un modelo matemático conformado por un tipo de datos y los métodos que operan sobre estos datos. Un ejemplo de un tipo de datos abstractos es el TDA Conjunto donde se puede definir el tipo de datos a utilizar, por ejemplo, utilizar números enteros o con decimales y sus operaciones ya conocidas de un conjunto: unión, intersección y diferencia.

Para realizar un programa utilizando un TDA se debe imaginar que se va a tener una división entre lo que es la implementación del TDA y la parte de la aplicación, por lo que se puede realizar cambios en el TDA sin que afecte la parte de la aplicación.



Ejemplo de TDA Punto

Para empezar con la implementación del TDA, se debe establecer cuáles son las clases de la implementación y cuáles de la aplicación. En este caso vamos a trabajar con 3 clases: la clase Punto donde se defina el tipo de datos a utilizar, la clase OperacionesPunto con los métodos para poder trabajar con el tipo de datos y la clase Menu que será la clase de aplicación.

Primero vamos a establecer un objeto llamado Punto con 2 variables: coordenadaX y coordenada Y, las 2 de tipo entero. Como es un objeto le definimos a cada variable sus métodos respectivos de set y get. (Ver Figura 01. Clase Punto)

```
1. public class Punto
2. {
3.     private int coordenadaX;
4.     private int coordenadaY;
5.
6.     public Punto()
7.     {
8.         coordenadaX = 0;
9.         coordenadaY = 0;
10.    }//Fin del constructor
11.
12.    public Punto (int x, int y)
13.    {
14.        coordenadaX = x;
15.        coordenadaY = y;
16.    }//Fin del constructor con parámetros
17. }
```

Figura 1: a. Clase Punto (constructores)

Tipos de datos abstractos

```

18. public int getCoordenadaX()
19. {
20.     return coordenadaX;
21. }//Fin de getCoordenadaX
22.
23. public void setCoordenadaX(int coordenadaX)
24. {
25.     this.coordenadaX = coordenadaX;
26. }//Fin de setCoordenadaX
27.
28. public int getCoordenadaY()
29. {
30.     return coordenadaY;
31. }//Fin de getCoordenadaY
32.
33. public void setCoordenadaY(int coordenadaY)
34. {
35.     this.coordenadaY = coordenadaY;
36. }//Fin de setCoordenadaY
37. }//Fin de la clase

```

Figura 1: a. Clase Punto (métodos set y get)

Segundo creamos la clase OperacionesPunto donde se encontrará los métodos que utilizarán la clase Punto, por ejemplo: distancia (), asignarValorAX (), asignarValorAY (), imprimirCoordenadas(), entre otros (Ver Figura 02, Clase OperacionesPunto).

```

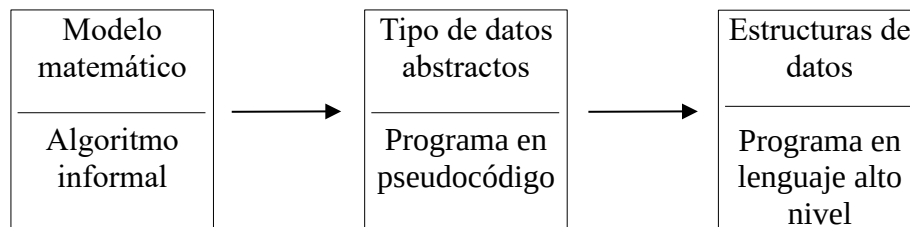
1. public class OperacionesPunto
2. {
3.     Punto punto = new Punto();
4.
5.     public int distancia (int valorX, int valorY) {      }
6.
7.     public void asignarValorAX (int valorX) {  }
8.
9.     public void asignarValorAY(int valorY) {  }
10.
11.     public String imprimirCoordenadas() {  }
12. }//Fin de la clase OperacionesPunto

```

Figura 2: Clase OperacionesPunto

Tipos de datos, estructuras de datos y TDAs

El proceso de desarrollo de un programa de aplicación puede ser interpretado en los siguientes pasos:



Es importante diferenciar entre los conceptos: TDA, tipo de dato y estructura de datos.

Tipo de datos:

El **tipo de datos** de una variable se define como el conjunto de valores que ésta puede tomar. Los tipos de datos básicos varían de un lenguaje a otro, por ejemplo en java son: entero, real, booleano, carácter, entre otros. También se pueden construir tipos de datos compuestos a partir de los básicos.

Estructura de datos:

El término **estructura de datos** se refiere a una colección de variables en memoria que están relacionadas de alguna forma específica. El componente básico de una estructura de datos es la celda. Se puede representar una celda como una caja capaz de almacenar un valor tomado de algún tipo de datos básico o compuesto. Las estructuras de datos se crean dando nombres a agregados de celdas y (opcionalmente) interpretando los valores de algunas celdas como representantes de conexiones entre celdas (por ejemplo, los apuntadores).

Ejemplos de estructuras de datos:

- Arreglos
- Registros
- Archivos
- Estructuras con apuntadores

Tipos de datos abstractos:

Como vimos al inicio, un TDA es un modelo matemático, junto con varias operaciones definidas sobre ese modelo. Los algoritmos deben diseñarse en función de los TDA, pero para implementar un algoritmo en un lenguaje de programación determinado debe hallarse alguna manera de representar los TDA en función de los tipos de datos y los operadores manejados por ese lenguaje. Para representar el

modelo matemático básico de un TDA, se emplean estructuras de datos, que son conjuntos de variables, quizás de tipo distintos, conectados entre sí de diversas formas.

Ventajas de implementar TDA

Un tipo de dato abstracto es un modelo (estructura) con un número de operaciones que afectan a ese modelo. Los tipos de datos abstractos proporcionan numerosos beneficios al programador, que se pueden resumir en los siguientes:

- Permiten una mejor conceptualización y modelización del mundo real. Mejoran la representación y la comprensibilidad. Clarifican los objetos basados en estructuras y comportamientos comunes.
- Mejoran la robustez del sistema. Permiten la especificación del tipo de cada variable. Los tipos de datos abstractos permiten la comprobación de tipos para evitar errores de tipo en tiempo de ejecución.
- Mejoran el rendimiento. Para sistemas tipeados (tipificados), el conocimiento de los objetos permite la optimización de tiempo de compilación.
- Separan la implementación de la especificación. Permiten la modificación y la mejora de la implementación sin afectar la interfaz pública del TDA.
- Permiten la extensibilidad del sistema.
- Utilizan mejor la semántica del tipo. Los TDA agrupan o localizan las operaciones y la representación de atributos.
- Al utilizar el encapsulamiento de datos facilita la modificación del tipo de datos. Según Joyanes (2008) “El principio de encapsulamiento significa que las estructuras de datos internas utilizadas en la implementación de una clase no pueden ser accesibles directamente al usuario de la clase”.
- Al encapsular debidamente el tipo de datos implementado, es más fácil reutilizarlo en otros programas que necesitan utilizar ese mismo TDA, por ser fácil de crear y mantener.

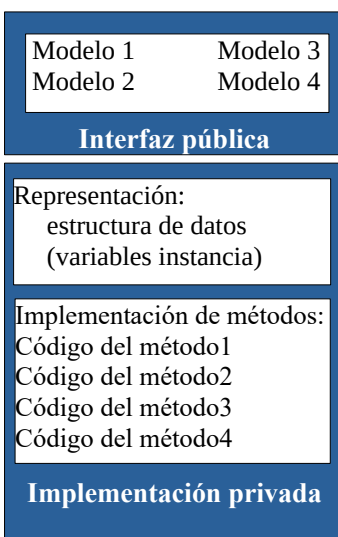
Estructura de un TDA

Algunos lenguajes de programación tienen características que nos permiten ampliar el lenguaje agregando sus propios tipos de datos. Como indicamos anteriormente, un tipo de dato creado por el programador se denomina tipo abstracto de dato (TDA) para diferenciarlo del tipo fundamental de datos.

Por ejemplo, en java, el tipo *Punto*, que representa la coordenada X y la coordenada Y de un sistema de coordenadas no existe. Sin embargo, es posible que implementemos el tipo abstracto de datos, considerando los valores que se almacenan en las variables definidas, así como las operaciones que están disponibles para manipular estas variables. En esencia, un tipo de dato abstracto es un tipo de dato que consta de datos (definidos como estructuras de datos propias) y operaciones que se pueden realizar sobre ellos. Por lo tanto, un TDA se compone de las estructuras de datos y los procedimientos que manipulan esas estructuras de datos.

La estructura de un tipo de dato abstracto (clase), desde un punto de vista global, se compone de la interfaz y de la implementación.

Las estructuras de datos elegidas para almacenar la representación de un tipo abstracto de datos son ocultas a los usuarios. Los algoritmos utilizados para implementar cada una de las operaciones de los tipos de datos abstractos están encapsulados dentro de los propios TDA. La característica de ocultamiento de la información significa que los objetos tienen interfaces que son *públicas*. Sin embargo, las implementaciones de esas interfaces son *privadas*.



Tipos de datos abstractos

Un programa que maneja un TDA lo hace teniendo en cuenta las operaciones que tiene, sin importarle la representación física de los datos. Es decir, los usuarios que implementan un tipo de dato abstracto se comunican con él a partir de la interfaz que da el TDA mediante las funciones de acceso creadas. Por lo tanto, podemos cambiarse la implementación del tipo de dato que estamos utilizando, sin que se vea afectado el programa que usa el TDA ya que para el programa esa información está oculta.

Implementación de los TDA

Dependiendo del lenguaje de programación que se esté utilizando, la implementación de los tipos de datos abstractos reciben diferentes nombres, en java se definen como clase.

En los lenguajes de programación, los programas que implementan TDA definen dos partes:

- *Especificación* del TDA: declara las operaciones y los datos a utilizar.
- *Implementación*: muestra el código fuente de las operaciones creadas, que a la vez permanece oculto al exterior del módulo.

Bibliografía

Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1988). Estructuras de datos y algoritmos (Vol. 1). Addison-Wesley Iberoamericana.

Birtwistle, G. M., Dahl, O. J., Myhrhaug, B., & Nygaard, K. (1973) SIMULA BEGIN. Filadelfia, Pensilvania. Auerbach Press

Cook, W. R. (2009, October). On understanding data abstraction, revisited. In Proceedings of the 24th ACM SIGPLAN conference on Object oriented programming systems languages and applications (pp. 557-572).

Guttag, J. V. (1975). The specification and application to programming of abstract data types. University of Toronto.

Joyanes, L., Martínez, I., & Zahonero, I. (2008). Estructuras de datos en java. McGraw Hill.

Liskov, B., & Zilles, S. (1974). Programming with abstract data types. ACM Sigplan Notices, 9(4), 50-59.